

Rapport de Projet – Mini Task Manager API

Module : DevOps – L3 Génie Logiciel **Auteur :** [Ton Nom] **Date :** 28 Février 2026

Table des matières

1. [Introduction](#)
 2. [Architecture du projet](#)
 3. [Environnement de développement](#)
 4. [Module task-core](#)
 5. [Module task-api](#)
 6. [Conteneurisation Docker](#)
 7. [Déploiement Ansible](#)
 8. [Livrables et preuves](#)
-

1. Introduction

Ce projet consiste à développer une API REST de gestion de tâches selon une approche DevOps complète, en intégrant : - **Maven** pour la gestion des dépendances et la publication d'artifacts - **Nexus Repository Manager** comme gestionnaire d'artifacts privé - **Spring Boot** pour l'API REST - **MySQL** comme base de données - **Docker** pour la conteneurisation - **Ansible** pour l'automatisation du déploiement sur une VM Ubuntu

2. Architecture du projet

```
tp_ansible/
├── task-core/                                # Bibliothèque métier (publié sur Nexus)
│   ├── pom.xml
│   └── src/main/java/sn/isi/l3gl/core/
│       ├── entity/Task.java
│       ├── enums/TaskStatus.java
│       ├── repository/TaskRepository.java
│       └── service/TaskService.java
├── task-api/                                # API REST Spring Boot
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/java/sn/isi/l3gl/api/
│       ├── TaskApiApplication.java
│       └── controller/TaskController.java
```

```

├─ ansible/
│   ├── inventory.ini      # Hôtes cibles
│   └─ playbook.yml       # Playbook de déploiement
└─ docker-compose.yml     # Nexus + MySQL (dev local)

```

Stack technique : | Composant | Technologie | |———|———| |
 Langage | Java 21 | | Framework | Spring Boot 3.2.0 | | Base de données |
 MySQL 8.0 | | Build | Maven | | Registry | Nexus 3.70.1 | | Conteneur |
 Docker (eclipse-temurin:17-jdk-jammy) | | Déploiement | Ansible | | VM
 cible | Ubuntu Server 24.04 |

3. Environnement de développement

3.1 Lancement de Nexus et MySQL via Docker

```

# docker-compose.yml
services:
  nexus:
    image: sonatype/nexus3:3.70.1
    container_name: nexus
    ports: ["8081:8081"]
    volumes: [nexus-data:/nexus-data]

  mysql:
    image: mysql:8.0
    container_name: mysql-task
    ports: ["3306:3306"]
    environment:
      MYSQL_DATABASE: task_db
      MYSQL_USER: task_user
      MYSQL_PASSWORD: task_pass
      MYSQL_ROOT_PASSWORD: root

docker-compose up -d

```

Vérification : Nexus accessible sur <http://localhost:8081>

3.2 Configuration Maven pour Nexus

Fichier `~/.m2/settings.xml` créé avec les credentials Nexus :

```

<servers>
  <server>
    <id>nexus-snapshots</id>
    <username>admin</username>
    <password>passer123</password>
  </server>
  <server>
    <id>nexus-releases</id>
    <username>admin</username>
    <password>passer123</password>
  </server>
</servers>

```

4. Module task-core

4.1 Structure

- **GroupId** : sn.isi.l3gl.core
- **ArtifactId** : task-core
- **Dépendances** : spring-boot-starter-data-jpa, mysql-connector-j

4.2 Entité Task

```
@Entity
@Table(name = "tasks")
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String description;
    @Enumerated(EnumType.STRING)
    private TaskStatus status;
}
```

4.3 Enum TaskStatus

```
public enum TaskStatus {
    TODO, IN_PROGRESS, DONE
}
```

4.4 Repository

```
public interface TaskRepository extends JpaRepository<Task, Long> {
    long countByStatus(TaskStatus status);
}
```

4.5 Service avec évolutions SNAPSHOT

Version	Méthode ajoutée	Commande de déploiement
0.0.1-SNAPSHOT	createTask()	mvn deploy
0.1.0-SNAPSHOT	listTasks()	mvn deploy
0.2.0-SNAPSHOT	updateStatus()	mvn deploy
0.3.0-SNAPSHOT	countCompletedTasks()	mvn deploy
0.3.0 (RELEASE)	—	mvn deploy

4.6 Configuration distributionManagement (pom.xml)

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>http://localhost:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
```

```
        <url>http://localhost:8081/repository/maven-snapshots/</url>
      </snapshotRepository>
    </distributionManagement>
```

4.7 Commits Git (task-core)

```
feat: add Task entity, TaskRepository and createTask()
feat: add listTasks() method
feat: add updateStatus() method
feat: add countCompletedTasks() and release 0.3.0
```

5. Module task-api

5.1 Structure

- **GroupId** : sn.isi.l3gl.api
- **ArtifactId** : task-api
- **Port** : 8085
- **Dépendance** : task-core:0.3.0 depuis Nexus

5.2 Application principale

```
    @SpringBootApplication(scanBasePackages = {"sn.isi.l3gl.api",
"sn.isi.l3gl.core"})
    @EntityScan(basePackages = "sn.isi.l3gl.core.entity")
    @EnableJpaRepositories(basePackages = "sn.isi.l3gl.core.repository")
    public class TaskApiApplication { ... }
```

Note : Les annotations @EntityScan et @EnableJpaRepositories sont nécessaires car les beans JPA sont dans un module externe (task-core).

5.3 Endpoints REST

Méthode	URL	Description
POST	/api/tasks	Créer une tâche (status=TODO auto)
GET	/api/tasks	Lister toutes les tâches
PUT	/api/tasks/{id}/status	Modifier le statut d'une tâche
GET	/api/tasks/done/count	Compter les tâches DONE

5.4 Tests des endpoints

```
# Créer une tâche
curl -X POST http://localhost:8085/api/tasks \
-H "Content-Type: application/json" \
-d '{"title":"Première tâche","description":"Test"}'
# → {"id":1,"title":"Première
tâche","description":"Test","status":"TODO"}
```

```
# Lister les tâches
curl http://localhost:8085/api/tasks

# Mettre à jour le statut
curl -X PUT "http://localhost:8085/api/tasks/1/status?status=DONE"

# Compter les tâches terminées
curl http://localhost:8085/api/tasks/done/count
# → 1
```

6. Conteneurisation Docker

6.1 Dockerfile

```
FROM eclipse-temurin:17-jdk-jammy

WORKDIR /app

COPY target/task-api-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8085

ENTRYPOINT ["java", "-jar", "app.jar"]
```

6.2 Build et push de l'image

```
# Build du JAR
cd task-api
mvn clean package -DskipTests

# Build de l'image Docker
docker build -t cheikhkanteye/task-api:latest .

# Push sur Docker Hub
docker login
docker push cheikhkanteye/task-api:latest
```

Image publiée : cheikhkanteye/task-api:latest

7. Déploiement Ansible

7.1 Prérequis

- Ansible installé sur la machine hôte
- Clé SSH copiée sur la VM : `ssh-copy-id -i ~/.ssh/vm_key.pub -p 2222 ubuntu-server@127.0.0.1`
- Port forwarding VirtualBox : `host:2222 → VM:22, host:8085 → VM:8085`

7.2 Inventory (inventory.ini)

```
[vm]
```

```
ubuntu_server ansible_host=127.0.0.1 ansible_port=2222 \
ansible_user=ubuntu-server \
ansible_ssh_private_key_file=~/.ssh/vm_key \
ansible_become_password=passer \
ansible_ssh_extra_args='-o StrictHostKeyChecking=no'
```

7.3 Playbook (playbook.yml)

Le playbook réalise les étapes suivantes sur la VM Ubuntu :

1. **Installation de Docker CE** (dépôt officiel Docker)
2. **Configuration du driver de stockage vfs** (compatibilité VirtualBox)
3. **Lancement du conteneur MySQL** (image mysql:8.0)
4. **Pull et lancement du conteneur task-api** (image cheikhkanteye/task-api:latest)
5. **Vérification** de l'endpoint /api/tasks (status HTTP 200)

```
# Lancement du playbook
ansible-playbook -i inventory.ini playbook.yml
```

7.4 Résultat attendu

```
PLAY RECAP
*****
ubuntu_server : ok=12  changed=8  unreachable=0  failed=0
```

8. Livrables et preuves

Screenshots à fournir

N°	Description
1	Nexus – liste des versions SNAPSHOT de task-core
2	Nexus – version RELEASE 0.3.0
3	Docker Hub – image cheikhkanteye/task-api:latest
4	docker ps sur la VM (conteneurs mysql-task et task-api)
5	curl http://localhost:8085/api/tasks depuis la VM
6	Résultat du playbook Ansible (PLAY RECAP ok=12 failed=0)

Commandes de vérification sur la VM

```
# Vérifier les conteneurs
docker ps

# Tester l'API
curl -X POST http://localhost:8085/api/tasks \
-H "Content-Type: application/json" \
-d '{"title":"Test Ansible","description":"Déployé via Ansible"}'

curl http://localhost:8085/api/tasks
```

Conclusion

Ce projet a permis de mettre en pratique une chaîne DevOps complète : - **Développement** incrémental avec versioning Maven (SNAPSHOT → RELEASE) - **Publication** d'artifacts sur Nexus privé - **Conteneurisation** de l'application avec Docker - **Automatisation** du déploiement avec Ansible

Les principaux défis rencontrés : - Configuration du scan de packages Spring Boot pour un module externe - Compatibilité du driver de stockage Docker dans un environnement VirtualBox - Installation correcte de la VM Ubuntu Server (démarrage depuis ISO vs installation sur disque)